Eskadra Bielik - Misja 2 - RAG w oparciu o model Bielik i Google Cloud

Suwerenne i wiarygodne AI - Od dokumentów firmowych do inteligentnej bazy wiedzy w oparciu o model Bielik i Google Cloud.

> [!WARNING] **Materiał warsztatowy — wyłącznie do celów edukacyjnych.** Kod i konfiguracja zawarte w tym repozytorium nie są przystosowane do wdrożeń produkcyjnych. Celowo pominięto m.in. uwierzytelnianie API, zarządzanie sekretami, monitoring oraz limity kosztów, aby uprościć przebieg warsztatu i skupić się na zrozumieniu architektury RAG.

Agenda warsztatu

#	Temat	Czas	Punkty
0	Wstęp — czym jest RAG, Bielik i architektura rozwiązania	15 min	—
1	Przygotowanie projektu Google Cloud	15 min	**5**
2	Konfiguracja zmiennych środowiskowych i usług Google Cloud	10 min	**10**
3	Uruchomienie modeli Bielik i EmbeddingGemma na Cloud Run (równolegle)	15 min	**20**
4	Inicjalizacja wektorowej bazy danych w BigQuery	5 min	**5**
5	Uruchomienie API (Orchestration) na Cloud Run	15 min	**10**
—	**Przerwa — lunch / poczęstunek / kawa / herbata / sok**	**20 min**	—
6	Testowanie API — zasilanie bazy i pierwsze zapytania RAG	10 min	**10**
7	Przegląd API i architektury kodu	5 min	**5**
8	Interfejs Web UI — porównanie modelu z RAG i bez RAG + eksperymenty	20 min	**10**
9	Certyfikat ukończenia warsztatu	5 min	—
10	Czyszczenie zasobów Google Cloud	5 min	—
11	Networking	15 min	—
	Łącznie	**~155 min**	**75 pkt**

Jak czytać ten przewodnik

Placeholdery — co wpisać zamiast `<...>`

Gdy w komendzie widzisz tekst ujęty w nawiasy ostre `<`, `>`, zastąp go swoją wartością **bez nawiasów**.

Zapis w przewodniku	Co wpisujesz zamiast tego

Zapis w przewodniku	Co wpisujesz zamiast tego
`gcloud config set project <ID_TWOJEGO_PROJEKTU>`	`gcloud config set project my-project-123`

> [!CAUTION] Nawiasy `<` i `>` są tylko znacznikiem miejsca — **nie wpisuj ich** do terminala. Wpisz wyłącznie swoją wartość.

Zmienne środowiskowe — `$NAZWA`

Gdy w komendzie widzisz `$PROJECT_ID`, `$REGION` lub `$ORCHESTRATION_URL` — **nie zmieniaj nic i nie przepisuj ręcznie**. Są one ustawiane automatycznie przez skrypt `setup_env.sh` i terminal sam podstawia właściwą wartość podczas wykonania komendy.

Zapis w przewodniku	Co terminal widzi w praktyce
`--region $REGION`	`--region europe-west1`
`--project $PROJECT_ID`	`--project my-project-123`
`"$ORCHESTRATION_URL/ask"`	`"https://twoja-usługa.run.app/ask"`

Podstawianie komendy — `$(komenda)`

Zapis `$(gcloud run services describe ...)` oznacza: uruchom komendę wewnątrz `$(...)` i użyj jej wyniku jako wartości. Możesz wkleić cały blok kodu bez żadnych modyfikacji.

O projekcie

Niniejsze repozytorium prezentuje kompletne, bezserwerowe (serverless) rozwiązanie klasy RAG (Retrieval-Augmented Generation) wdrożone w chmurze Google Cloud. Głównym celem aplikacji jest dostarczenie wydajnego i suwerennego inteligentnego asystenta zdolnego do odpowiadania na pytania użytkownika w oparciu o dedykowaną bazę wiedzy (np. wewnętrzne dokumenty, regulaminy).

Podstawowa architektura wdrażanego rozwiązania opiera się na poniższych serwisach i komponentach:

- **Modelu językowym LLM:** Suwerenny polski model Bielik charakteryzujący się bardzo dobrym zrozumieniem języka polskiego oraz polskiego kontekstu kulturowego. Uruchomiony w usłudze Cloud Run, odpowiada za ostateczne generowanie naturalnej dla użytkownika odpowiedzi.
- **Modelu osadzania (Embedding):** Wydajny model EmbeddingGemma uruchomiony w usłudze Cloud Run, służący do szybkiej zamiany tekstu (zapytań użytkownika i dokumentów docelowych) na reprezentację wektorową.
- **Wektorowej Bazie Wiedzy:** Skalowalna hurtownia danych BigQuery z mechanizmem Vector Search zapewniająca wektorowe wyszukiwanie semantycznie dopasowanych fragmentów z pośród milionów dokumentów źródłowych.
- **Logice i serwerze aplikacyjnym:** Aplikacja napisana w języku Python (z frameworkiem FastAPI), udostępniająca nakładkę graficzną Web UI oraz publiczne API spinające platformy w całość.

Dodatkowo, dzięki prostemu interfejsowi graficznemu, aplikacja pozwala na wygodne porównanie i empiryczne przetestowanie "surowego" modelu Bielik polegającego tylko na sobie w konfrontacji z

bogatszą odpowiedzią modelu wspartego kontekstem RAG.

> [!TIP] Jeśli chcesz lepiej zrozumieć ideę RAG przed przystąpieniem do warsztatu, zapoznaj się z wprowadzeniem Google Cloud: Retrieval-Augmented Generation

Diagramy architektury

Szczegółowe diagramy i dokumentacja architektoniczna dostępne są w katalogu `architektura/`:

Plik	Zawartość
`01_widok_systemowy.md`	Pełna mapa komponentów GCP i przepływu danych
`02_pipeline_rag.md`	Diagram sekwencji endpointu `/ask` (RAG)
`03_pipeline_ingestion.md`	Diagram sekwencji endpointu `/ingest`
`04_kroki_warsztatu.md`	Kolejność budowania systemu krok po kroku
`05_mapa_repozytorium.md`	Struktura plików i ich rola w architekturze
`prompty_nano_banana.md`	Prompty dla agentów AI — spójność architektoniczna

Z czego składa się kod?

Przykładowy kod źródłowy zawarty w tym repozytorium pozwala w szczególności na:

- Skonfigurowanie własnej instancji modelu Bielik w oparciu o silnik Ollama
- Skonfigurowanie własnej instancji modelu osadzającego (embedding model) EmbeddingGemma w oparciu o Ollama
- Uruchomienie obu powyższych modeli na platformie typu bezserwerowego: Cloud Run
- Skonfigurowanie bazy wektorów w BigQuery wraz ze specjalnym zaawansowanym przeszukiwaniem BigQuery Vector Search

1. Przygotowanie projektu Google Cloud `~15 min`

Krok 1.1 — Aktywacja konta rozliczeniowego z kredytami OnRamp

> [!NOTE] Kredyty OnRamp pozwalają korzystać z Google Cloud **bez karty kredytowej**. Otrzymasz od prowadzącego indywidualny link do aktywacji kredytów.

1. Otwórz otrzymany od prowadzącego link do aktywacji kredytów i postępuj zgodnie z instrukcjami

> [!TIP] Szczegółową instrukcję aktywacji kredytów znajdziesz w tym przewodniku: Google Cloud Credits Redemption

2. Wypełnij formularz aktywacji — podaj imię i nazwisko, zaakceptuj regulamin
3. Potwierdź że konto rozliczeniowe zostało aktywowane — pojawi się komunikat o przyznaniu kredytów

Krok 1.2 — Utworzenie nowego projektu Google Cloud

1. Przejdź do selektora projektów w górnym pasku Google Cloud Console i kliknij **Nowy projekt**

> [!TIP] Szczegółową instrukcję tworzenia projektu znajdziesz w tym przewodniku: Google Cloud Credits Redemption — krok 2

2. Nadaj projektowi nazwę (np. `bielik-warsztat`) i jako konto rozliczeniowe wybierz konto aktywowane w poprzednim kroku
3. Kliknij **Utwórz** i poczekaj aż projekt zostanie utworzony
4. Upewnij się że nowo utworzony projekt jest aktywny (widoczny w selektorze projektów w górnym pasku)

> [!CAUTION] Nie pomyl nazwy projektu z ID projektu — nie zawsze są takie same. ID projektu widoczne jest pod nazwą podczas tworzenia i na stronie głównej konsoli.

> [!TIP] Możesz potwierdzić że kredyty są powiązane z projektem wchodząc w menu po lewej stronie: **Billing → Credits**

Krok 1.3 — Otwarcie terminala Cloud Shell i sklonowanie repozytorium

1. Otwórz terminal Cloud Shell klikając ikonę `>_` w górnym pasku Google Cloud Console (dokumentacja)
2. Zweryfikuj że zalogowane jest właściwe konto

```
gcloud auth list
```

> [!TIP] Jeżeli widoczne jest inne konto niż to z kredytami, zaloguj się komendą: `gcloud auth login`

3. Potwierdź że aktywny jest właściwy projekt

```
gcloud config get project
```

> [!TIP] Jeżeli projekt jest inny niż oczekiwany, zmień go komendą: `gcloud config set project <ID_TWOJEGO_PROJEKTU>`

4. Sklonuj repozytorium z kodem warsztatu

```
git clone https://github.com/Legard777/eskadra-bielik-misja2
```

5. Przejdź do katalogu z kodem

```
cd eskadra-bielik-misja2
```

> [!TIP] Cloud Shell posiada wbudowany edytor graficzny — przydatny do przeglądania i edycji plików bez znajomości edytorów terminalowych. Na potrzeby tego warsztatu nie jest wymagany, jednak możesz go uruchomić w dowolnym momencie komendą `cloudshell workspace .` lub klikając przycisk **Open Editor** w górnym pasku Cloud Shell. Więcej informacji: Cloud Shell Editor

6. Zalicz krok i zdobądź **+5 punktów** — uruchom skrypt weryfikacyjny:

```
./checkpoints/checkpoint_1.sh
```

2. Konfiguracja zmiennych środowiskowych i usług Google Cloud `~10 min`

1. Przejrzyj zawartość skryptu `setup_env.sh`

```
cat setup_env.sh
```

> [!TIP] **Zadanie dla Gemini CLI** — zamiast czytać opis, zapytaj AI! Uruchom w terminalu:
>
> ```
> gemini "Co robi ten skrypt @setup_env.sh? Wyjaśnij każdą zmienną środowiskową."
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja może brzmieć zupełnie inaczej i to jest jak najbardziej w porządku. Modele językowe są niedeterministyczne: za każdym razem generują odpowiedź od nowa, dlatego dwie osoby zadające to samo pytanie mogą otrzymać różne, ale równie poprawne wyjaśnienia.

2. Uruchom skrypt `setup_env.sh`

```
source setup_env.sh
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI o różnicę między `source` a `./`:
>
> ```
> gemini "Jaka jest różnica między source setup_env.sh a ./setup_env.sh w bashu? Kiedy używać każdej z form?"
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja wersja może być krótsza, dłuższa lub podać inne przykłady. Właśnie tak działają modele językowe.

> [!IMPORTANT] Jeżeli z jakiegoś powodu musisz ponownie uruchomić terminal Cloud Shell, pamiętaj aby ponownie uruchomić skrypt `setup_env.sh` aby wczytać zmienne

> środowiskowe.

3. Uruchom skrypt ochrony plików źródłowych *(tylko raz — zabezpiecza pliki `.py`, `.html`, `.csv` przed przypadkową edycją)*

```
./protect_files.sh
```

4. Włącz potrzebne usługi w projekcie Google Cloud

```
gcloud services enable run.googleapis.com
gcloud services enable cloudbuild.googleapis.com
gcloud services enable artifactregistry.googleapis.com
gcloud services enable bigquery.googleapis.com
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI do czego służą te komendy:
>
> ```
> gemini "Do czego służy komenda gcloud services enable? Wyjaśnij po krótce każdą z usług: run, cloudbuild, artifactregistry, bigquery."
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym.

5. Uzyskaj uprawnienia do wywoływania usług Cloud Run

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
 --member=user:$(gcloud config get-value account) \
 --role='roles/run.invoker'
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI o uprawnienia IAM w Google Cloud:
>
> ```
> gemini "Co robi komenda gcloud projects add-iam-policy-binding? Wyjaśnij czym jest rola roles/run.invoker i dlaczego zasada najmniejszych uprawnień jest ważna."
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym.

6. Zalicz krok i zdobądź **+10 punktów** — uruchom skrypt weryfikacyjny:

```
./checkpoints/checkpoint_2.sh
```

3. Uruchomienie modeli LLM Bielik i EmbeddingGemma na Cloud Run ~15 min

Poniższe kroki przeprowadzą Cię przez wdrożenie obu modeli **jeden po drugim** w tym samym terminalu.

> [!TIP] **Dla bardziej doświadczonych:** wdrożenia obu modeli można uruchomić równolegle w dwóch osobnych terminalach Cloud Shell, co skróci czas oczekiwania. Jeśli wiesz jak to zrobić — śmiało! Możesz przełączać się między terminalami klikając ich zakładki w dolnym pasku Cloud Shell. Pamiętaj aby w każdym nowym terminalu wczytać zmienne środowiskowe (`source ~/eskadra-bielik-misja2/setup_env.sh`) i poczekać aż oba wdrożenia zakończą się sukcesem zanim przejdziesz do następnego kroku. Jeśli nie — wykonaj kroki po kolei zgodnie z instrukcją poniżej.

Model LLM Bielik

1. Przejrzyj zawartość skryptu `llm/cloud_run.sh`

```
cat llm/cloud_run.sh
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI co robi ten skrypt:
>
> ```
> gemini "Co robi ten skrypt @llm/cloud_run.sh? Wyjaśnij każdą flagę komendy gcloud run deploy."
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja może brzmieć zupełnie inaczej i to jest jak najbardziej w porządku. Modele językowe są niedeterministyczne: za każdym razem generują odpowiedź od nowa.

2. Uruchom skrypt wdrożeniowy

```
cd llm
./cloud_run.sh
```

3. Sprawdź czy usługa `bielik` pojawiła się w Cloud Console → Cloud Run → Services i ma status **Ready**

4. Przejrzyj zawartość pliku `llm/llm_test1.sh`

```
cat llm_test1.sh
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI co robi ten skrypt:

> ```
> gemini "Co robi ten skrypt @llm/llm_test1.sh? Wyjaśnij jak działa autoryzacja i co oznaczają poszczególne pola w ciele zapytania JSON."
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja może brzmieć zupełnie inaczej i to jest jak najbardziej w porządku. Modele językowe są niedeterministyczne: za każdym razem generują odpowiedź od nowa.

5. Zadaj pierwsze pytanie modelowi Bielik

```
./llm_test1.sh
```

6. Wróć do głównego katalogu projektu

```
cd ..
```

Model EmbeddingGemma

1. Przejrzyj zawartość skryptu `embedding_model/cloud_run.sh`

```
cat embedding_model/cloud_run.sh
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI co robi ten skrypt:
>
> ```
> gemini "Co robi ten skrypt @embedding_model/cloud_run.sh? Wyjaśnij każdą flagę komendy gcloud run deploy. Czym różni się ta konfiguracja od wdrożenia modelu LLM?"
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja może brzmieć zupełnie inaczej i to jest jak najbardziej w porządku. Modele językowe są niedeterministyczne: za każdym razem generują odpowiedź od nowa.

2. Uruchom skrypt wdrożeniowy

```
cd embedding_model
./cloud_run.sh
```

3. Sprawdź czy usługa `embedding-gemma` pojawiła się w Cloud Console → Cloud Run → Services i ma status **Ready**

4. Przejrzyj zawartość pliku `embedding_model/embedding_test1.sh`

```
cat embedding_test1.sh
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI co robi ten skrypt:
>
> ```
> gemini "Co robi ten skrypt @embedding_model/embedding_test1.sh? Czym różni się endpoint /api/embed od /api/chat i czym są embeddingi (wektory)?"
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja może brzmieć zupełnie inaczej i to jest jak najbardziej w porządku. Modele językowe są niedeterministyczne: za każdym razem generują odpowiedź od nowa.

5. Wygeneruj pierwsze testowe embeddingi (wektory) dla przykładowego tekstu

```
./embedding_test1.sh
```

6. Wróć do głównego katalogu projektu

```
cd ..
```

7. Zalicz krok i zdobądź **+20 punktów** — oba modele wdrożone, to najtrudniejszy etap warsztatu:

```
./checkpoints/checkpoint_3.sh
```

4. Inicjalizacja wektorowej bazy danych w BigQuery `~5 min`

Projekt wykorzystuje BigQuery z funkcją Vector Search jako bazę z wiedzą kontekstową.

1. Przejdź do katalogu `vector_store`

```
cd vector_store
```

2. Zainstaluj wymagane biblioteki

```
pip install google-cloud-bigquery
```

> [!NOTE] Celowo pomijamy tworzenie wirtualnego środowiska Python (`venv`). W warsztacie korzystamy z Cloud Shell, który jest tymczasowym środowiskiem uruchamianym od nowa po każdej sesji — instalacja globalna jest tu w zupełności wystarczająca. Wirtualne środowisko byłoby przydatne przy długotrwałym projekcie, gdzie chcemy izolować zależności między aplikacjami na tej samej maszynie.

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI czym jest ta biblioteka i do czego służy:
>
> ```
> gemini "Do czego służy biblioteka google-cloud-bigquery w Pythonie? Czym jest pip i dlaczego w tym przypadku nie potrzebujemy wirtualnego środowiska venv?"
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`.

3. Przejrzyj kod skryptu inicjalizacyjnego

```
cat init_db.py
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI co robi ten skrypt:
>
> ```
> gemini "Co robi ten skrypt @vector_store/init_db.py? Wyjaśnij schemat tabeli BigQuery i dlaczego pole embedding ma typ FLOAT64 REPEATED."
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja może brzmieć zupełnie inaczej i to jest jak najbardziej w porządku. Modele językowe są niedeterministyczne: za każdym razem generują odpowiedź od nowa.

4. Uruchom skrypt inicjalizacyjny, który stworzy zbiór danych i tabelę w BigQuery

```
python init_db.py
```

5. Wróć do głównego katalogu projektu

```
cd ..
```

6. Zalicz krok i zdobądź **+5 punktów** — uruchom skrypt weryfikacyjny:

```
./checkpoints/checkpoint_4.sh
```

5. Uruchomienie API (Orchestration) na Cloud Run `~15 min`

Aplikacja Orchestration to serce całego rozwiązania RAG — spina model embeddingowy, BigQuery Vector Search i model Bielik w jeden przepływ i udostępnia go przez API oraz interfejs Web UI.

1. Przejrzyj kod aplikacji FastAPI

```
cat orchestration/main.py
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI co robi ta aplikacja:
>
> ```
> gemini "Co robi plik @orchestration/main.py? Wyjaśnij każdy endpoint API i opisz przepływ danych w endpoincie /ask: od zapytania użytkownika do odpowiedzi modelu Bielik."
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym — Twoja może brzmieć zupełnie inaczej i to jest jak najbardziej w porządku. Modele językowe są niedeterministyczne: za każdym razem generują odpowiedź od nowa.

2. Przejrzyj skrypt wdrożeniowy

```
cat orchestration/cloud_run.sh
```

> [!TIP] **Zadanie dla Gemini CLI** — zapytaj AI co robi ten skrypt:
>
> ```
> gemini "Co robi ten skrypt @orchestration/cloud_run.sh? Dlaczego musi najpierw pobrać URL-e modeli i jak przekazuje je do aplikacji przez zmienne środowiskowe?"
> ```
>
> W celu zamknięcia Gemini CLI wybierz komendę `/quit`. Porównaj swoją odpowiedź z opisem referencyjnym.

3. Przejdź do katalogu `orchestration`

```
cd orchestration
```

4. Uruchom skrypt wdrażający aplikację na Cloud Run

```
./cloud_run.sh
```

5. Po zakończeniu wdrożenia pobierz adres URL usługi i zapisz go do zmiennej środowiskowej

```
export ORCHESTRATION_URL=$(gcloud run services describe orchestration-
api --region $REGION --format="value(status.url)")
```

> [!NOTE] Zmienna `$ORCHESTRATION_URL` będzie potrzebna w kolejnych krokach do wysyłania zapytań przez `curl`. Jak wszystkie zmienne środowiskowe — działa tylko w bieżącym terminalu.

6. Wróć do głównego katalogu

```
cd ..
```

7. Zalicz krok i zdobądź **+10 punktów** — uruchom skrypt weryfikacyjny:

```
./checkpoints/checkpoint_5.sh
```

☕ Przerwa — lunch / poczęstunek / kawa / herbata / sok `~20 min`

> Wszystkie komponenty są wdrożone i gotowe. Po przerwie przetestujemy całe rozwiązanie RAG w akcji.

> [!IMPORTANT] **Powrót po przerwie — sprawdź terminal przed kontynuacją.** Cloud Shell automatycznie rozłącza się po okresie bezczynności, co usuwa wszystkie zmienne środowiskowe z pamięci. Jeśli robiłeś przerwę, uruchom poniższe komendy przed przejściem do kroku 6:
>
> ```
> source ~/eskadra-bielik-misja2/setup_env.sh
> ```
>
> ```
> export ORCHESTRATION_URL=$(gcloud run services describe
> orchestration-api --region $REGION --format="value(status.url)")
> ```
>
> Jeśli nie robiłeś przerwy i terminal był aktywny — możesz pominąć ten krok.

---

6. Testowanie API — Zasilanie i Wyszukiwanie (RAG) `~10 min`

1. Przejrzyj plik z przykładowymi danymi

```
tr -d '\r' < vector_store/hotel_rules.csv | awk -F',' '{printf "%-4s
%s\n", $1, $2}'
```

Plik CSV zawiera dwie kolumny:

Kolumna	Opis
`id`	Unikalny identyfikator rekordu
`text`	Treść dokumentu — zasada hotelowa w języku naturalnym

> [!NOTE] Po wgraniu danych przez endpoint `/ingest` aplikacja automatycznie doda trzecią kolumnę: **`embedding`** — wygenerowany przez EmbeddingGemma wektor liczbowy reprezentujący znaczenie tekstu. To właśnie ta kolumna umożliwia semantyczne wyszukiwanie w BigQuery Vector Search.

2. Wgraj przykładowe dane do BigQuery z pliku CSV

```
curl -X POST "$ORCHESTRATION_URL/ingest" \
     -F "file=@vector_store/hotel_rules.csv"
```

3. Zweryfikuj czy rekordy pojawiły się w BigQuery

   Otwórz BigQuery w Google Cloud Console, przejdź do tabeli `rag_dataset` → `hotel_rules` i kliknij przycisk **Preview** aby podejrzeć dane.

> [!TIP] **Preview jest bezpłatny** — nie wykonuje zapytania SQL i nie zużywa limitu darmowych zapytań BigQuery. To najszybszy sposób sprawdzenia czy dane zostały załadowane poprawnie.

> [!NOTE] Dane tekstowe w kolumnach `id`, `content` widoczne są natychmiast. Indeksowanie kolumny `embedding` na potrzeby Vector Search może chwilę potrwać — to normalne i nie blokuje kolejnych kroków.

> [!TIP] **Dla chętnych — weryfikacja SQL:** jeśli chcesz zobaczyć dane zapytaniem, wklej w edytorze BigQuery:
>
> ```
> SELECT id, content, ARRAY_LENGTH(embedding) AS
> embedding_dimensions
> FROM `rag_dataset.hotel_rules`
> ORDER BY id
> LIMIT 10
> ```
>
> Kolumna `embedding_dimensions` pokaże ile wymiarów ma wygenerowany wektor.

4. Wykonaj testowe zapytania RAG

Pytanie o częstotliwość pomiaru chloru w basenie:

```
curl -X POST "$ORCHESTRATION_URL/ask" \
     -H "Content-Type: application/json" \
     -d '{"query": "Jak często powinien być mierzony poziom chloru w basenie?"}'
```

Pytanie o godzinę podawania śniadania:

```
curl -X POST "$ORCHESTRATION_URL/ask" \
     -H "Content-Type: application/json" \
     -d '{"query": "O której godzinie jest podawane śniadanie?"}'
```

Pytanie o parking:

```
curl -X POST "$ORCHESTRATION_URL/ask" \
     -H "Content-Type: application/json" \
     -d '{"query": "Ile kosztuje parking hotelowy?"}'
```

> [!TIP] **Dla chętnych — odpowiednik SQL:** każde z powyższych zapytań wewnętrznie wykonuje Vector Search w BigQuery. Możesz zobaczyć jak to wygląda „pod maską", wklejając w edytorze BigQuery (zastąp `[...]` wektorem zwróconym przez `/api/embed`):
>
> ```
> SELECT base.content, distance
> FROM VECTOR_SEARCH(
>  TABLE `rag_dataset.hotel_rules`,
>  'embedding',
>  (SELECT [...] AS embedding),
>  top_k => 3,
>  distance_type => 'COSINE'
> )
> ```
>
> Wynik to 3 dokumenty semantycznie najbliższe zapytaniu — dokładnie to, co aplikacja wysyła jako kontekst do modelu Bielik.

5. Zalicz krok i zdobądź **+10 punktów** — uruchom skrypt weryfikacyjny:

```
./checkpoints/checkpoint_6.sh
```

7. Interfejs Programistyczny (API) ~5 min

Aplikacja udostępnia proste API stworzone przy pomocy frameworka *FastAPI*, pozwalające nie tylko na zasilanie bazy wiedzy, ale również na zadawanie pytań.

Aplikacja definiuje w pliku `orchestration/main.py` następujące ścieżki:

- `GET /` – serwuje statyczny plik interfejsu użytkownika (`index.html`).
- `POST /ingest` – przyjmuje plik CSV i indeksuje zawarte w nim informacje jako wektory w BigQuery (wykorzystując model embeddingowy `EmbeddingGemma`).
- `POST /ask` – główny endpoint RAG:
  - zamienia zapytanie z tekstu na wektor,
  - wyszukuje semantycznie 3 najbardziej zbliżone dokumenty wektorowe w tabeli BigQuery,
  - buduje prompt z odnalezionym kontekstem,
  - wysyła połączony prompt do modelu `Bielik` i zwraca ostateczną odpowiedź wraz z wybranym i wykorzystanym kontekstem.
- `POST /ask_direct` – służy jako zestawienie porównawcze (baseline). Przyjmuje zapytanie i wysyła je bezpośrednio do bazowego modelu `Bielik`, z całkowitym pominięciem RAG.

Zalicz krok i zdobądź **+5 punktów** — uruchom skrypt weryfikacyjny, który potwierdzi że wszystkie usługi działają razem:

```
./checkpoints/checkpoint_7.sh
```

8. Interfejs Użytkownika (Web UI) `~20 min`

Oprócz interfejsu API, aplikacja udostępnia również prostą nakładkę WWW. Całość pozwala na wygodne sprawdzenie i porównanie działania bazowego modelu Bielik z modelem Bielik wspartym przez RAG.

Interfejs użytkownika zaimplementowano w jednym, statycznym pliku: `orchestration/static/index.html`.

Skrypt osadzony w pliku HTML wysyła dwa jednoczesne żądania do endpointów `/ask` (wsparty RAG) oraz `/ask_direct` (bezpośrednio do modelu `Bielik`) i prezentuje obie odpowiedzi modelu obok siebie celem zilustrowania różnic. Wyświetla obok również jakich dokładnie fragmentów dokumentów BigQuery model użył w przypadku posiłkowania się dodatkowym kontekstem RAG.

> [!TIP] Zachęcamy Cię gorąco do eksperymentów! Przejrzyj kod źródłowy plików `orchestration/main.py` oraz `orchestration/static/index.html`, aby zobaczyć, w jak prosty sposób w Pythonie łączy się wyszukiwanie wektorowe BigQuery z modelem LLM i serwuje dla prostej graficznej nakładki JavaScript.
>
> ```
> cat orchestration/main.py
> cat orchestration/static/index.html
> ```
>
> Spróbuj zmodyfikować instrukcję systemową w pliku `main.py`, aby polecić Bielikowi zachowywanie się jak pirat lub ekspert od IT! Najpierw odblokuj plik do edycji, a następnie otwórz go w edytorze Cloud Shell:

```
chmod +w orchestration/main.py
cloudshell edit orchestration/main.py
```

Uruchomienie interfejsu

Aby otworzyć interfejs graficzny testowej aplikacji z poziomu Twojego projektu:

1. Wyświetl i kliknij w adres URL usługi `orchestration-api` uruchamiając w terminalu poniższą komendę:

   ```
   echo $ORCHESTRATION_URL
   ```

2. Po otwarciu opublikowanej strony w Twojej przeglądarce internetowej, wpisz w okno dialogowe dowolne zapytanie (np. "Do której godziny jest otwarty basen?") i kliknij "Zapytaj".

3. Porównaj strumień odpowiedzi wyświetlany dla samej bazy wiedzy modelu (bez dodatkowego kontekstu) z bogatszą odpowiedzią RAG wygenerowaną w oparciu o wiedzę z przeszukiwania BigQuery Vector Search.

4. Zalicz krok i zdobądź **+10 punktów** — uruchom skrypt weryfikacyjny:

   ```
   ./checkpoints/checkpoint_8.sh
   ```

9. Certyfikat ukończenia warsztatu `~5 min`

Gratulacje — warsztat dobiegł końca! Wygeneruj zaszyfrowany certyfikat zawierający wszystkie 8 checkpointów i prześlij go prowadzącemu.

> [!IMPORTANT] Przed wygenerowaniem certyfikatu upewnij się, że wszystkie 8 checkpointów zostało wykonanych (pliki `cert_artifacts/checkpoint_N.enc` muszą istnieć). Skrypt sam to weryfikuje i zgłosi brakujące kroki.

```
./checkpoints/certyfikat_generate.sh
```

Po pomyślnym wykonaniu wyślij prowadzącemu plik:

```
cert_artifacts/checkpoint_certyfikat.enc
```

> [!TIP] Plik jest zaszyfrowany — możesz go przesłać przez dowolny kanał (email, formularz, Slack). Zawiera potwierdzenie wykonania wszystkich etapów warsztatu powiązane z Twoim kontem Google Cloud i projektem.

10. Czyszczenie zasobów Google Cloud `~5 min`

Po zakończeniu warsztatu usuń utworzone zasoby aby uniknąć niepotrzebnych kosztów.

Skrypt `cleanup.sh` usuwa wszystkie zasoby utworzone podczas warsztatu:

Zasób	Nazwa	Koszty	Uwagi
Cloud Run	`bielik`	naliczane przez czas działania instancji	
Cloud Run	`embedding-gemma`	naliczane przez czas działania instancji	
Cloud Run	`orchestration-api`	naliczane przez czas działania instancji	
BigQuery	dataset `rag_dataset`	w ramach free tier	
Artifact Registry	`cloud-run-source-deploy`	**~$0.01/miesiąc** za przechowywanie obrazów Docker	pojawia się w billingu nawet po zakończeniu warsztatu — należy usunąć
Cloud Storage	`run-sources-{PROJECT_ID}-{REGION}`	nie pojawia się w billingu	archiwa zip tworzone automatycznie przez `gcloud run deploy --source`

1. Wróć do głównego katalogu projektu i uruchom skrypt czyszczący

```
cd ~/eskadra-bielik-misja2
./cleanup.sh
```

2. Skrypt wyświetli listę zasobów do usunięcia i poprosi o potwierdzenie. Wpisz `tak` aby kontynuować.

3. Po zakończeniu zweryfikuj w Google Cloud Console, że zasoby zostały usunięte:

   - **Cloud Run:** console.cloud.google.com/run
   - **BigQuery:** console.cloud.google.com/bigquery
   - **Artifact Registry:** console.cloud.google.com/artifacts

Orientacyjny koszt warsztatu

Na podstawie rzeczywistego przebiegu warsztatu całkowity koszt wynosi **~$3–4**.

Dominującą pozycją jest GPU NVIDIA L4 używany przez model Bielik na Cloud Run. Usługi Cloud Run z GPU działają w trybie **instance-based billing** (wymagane przez Google Cloud) — oznacza to, że płacisz za każdą sekundę gdy instancja jest aktywna, niezależnie od tego czy w danej chwili obsługuje zapytanie. Instancja może skalować do zera gdy przez dłuższy czas nikt jej nie odpytuje, jednak ze względu na długi czas zimnego startu (ładowanie modelu) pozostaje aktywna przez cały czas trwania warsztatu.

Usługa	Składnik	Orientacyjny koszt
Cloud Run (Bielik)	GPU NVIDIA L4	~$1.30
Cloud Run	CPU — billing instancyjny	~$1.01
Cloud Run	RAM — billing instancyjny	~$0.25
Cloud Run	CPU — billing requestowy	~$0.03
Networking	Network Intelligence Center	~$0.02
Artifact Registry	Przechowywanie obrazów Docker	~$0.01/mies.
Łącznie		**~$3.91**

> [!IMPORTANT] Uruchom skrypt `cleanup.sh` niezwłocznie po zakończeniu warsztatu. Usługi Cloud Run z GPU naliczają koszty przez cały czas działania instancji — nawet gdy nikt z nich aktywnie nie korzysta.

Optymalizacje dla środowisk produkcyjnych

Konfiguracja użyta w tym warsztacie jest celowo uproszczona. Dla zastosowań produkcyjnych Google Cloud dokumentuje szereg optymalizacji:

- **Szybszy cold start** — pobieranie modelu z Cloud Storage zamiast osadzania go w obrazie Docker, użycie formatu GGUF
- **Niższe koszty GPU** — konfiguracja `min-instances: 0` + odpowiednie proby startowe, aby instancja skalowała do zera gdy nikt nie korzysta
- **Wyższa przepustowość** — tuning współbieżności (`concurrency`) i rozmiaru okna kontekstu modelu
- **Sieć** — Direct VPC z `egress: all-traffic` dla niższych opóźnień

Szczegóły: Cloud Run GPU Best Practices